Saiyed Shizain | CU24250233 | Roll: 48
BTech CSE (A) | 3rd Yr - 5th Sem | 2024-2028

Full-Stack Web Development — BTCS303P

LAB SHEET — 10: CampusConnect Full System (Tasks 2–7)

What Lab 10 adds over Lab 9 (documented)

Lab 9 shipped the secure API core. This sheet adds: Socket.io realtime (Task 2), Redis caching + benchmark (Task 3), the Redux frontend (Task 4), RTL tests (Task 6) and Docker Compose (Task 7). Shared core files are reused, not duplicated in this document; new/changed files are shown in full.

Task 2 — Real-time notifications (Socket.io)

Aim. JWT-authenticated sockets in role rooms; admin announcements fan out to students instantly.

Theory. Handshake auth reuses the REST token; rooms give one-call broadcast; the client auto-reconnects and bumps a Redux badge.

```
const { Server } = require("socket.io");
const { verifyAccessToken } = require("../utils/jwt");

function initSocket(httpServer, corsOrigin) {
  const io = new Server(httpServer, {
    cors: { origin: corsOrigin, credentials: true },
    connectionStateRecovery: { maxDisconnectionDuration: 2 * 60 * 1000 },
  });

  // Authenticate every socket with the same JWT access token used for REST.
  io.use((socket, next) => {
    try {
      const token = socket.handshake.auth && socket.handshake.auth.token;
      if (!token) return next(new Error("Unauthorized: missing token"));
      const payload = verifyAccessToken(token);
      socket.user = payload;
      return next();
    } catch (err) {
      return next(new Error("Unauthorized: invalid or expired token"));
    }
  });

  io.on("connection", (socket) => {
    // Role rooms let us broadcast "new-announcement" to all students at once.
    socket.join(socket.user.role);
    socket.join(`user:${socket.user.sub}`);
    console.log(`[socket] connected: ${socket.user.email} (${socket.user.role})`);
    socket.on("disconnect", (reason) => {
      console.log(`[socket] disconnected: ${socket.user.email} (${reason})`);
    });
  });

  return io;
}

module.exports = initSocket;
```

```
import { io } from "socket.io-client";
import { store } from "./app/store";
import { announcementReceived } from "./features/events/eventsSlice";

let socket = null;

// Authenticated socket: JWT travels in the handshake `auth` payload and is
```

```
// re-sent automatically on every reconnection attempt.
export function connectSocket() {
  const token = store.getState().auth.accessToken;
  if (!token) return null;
  socket = io(import.meta.env.VITE_SOCKET_URL || "http://localhost:5000", {
    auth: { token },
    reconnection: true,
    reconnectionAttempts: Infinity,
    reconnectionDelay: 1000,
    reconnectionDelayMax: 5000,
  });
  socket.on("connect", () => console.log("[socket] connected", socket.id));
  socket.on("disconnect", (reason) => console.log("[socket] disconnected:", reason));
  socket.on("connect_error", (err) => console.log("[socket] connect_error:", err.message
));
  socket.on("new-announcement", (announcement) => {
    store.dispatch(announcementReceived());
    console.log("[socket] new-announcement received:", announcement.title);
  });
  return socket;
}

export function disconnectSocket() {
  if (socket) socket.disconnect();
}
```

```
[socket] connected (server log on client connect)
[socket] new-announcement received: <title> (client log on admin POST)
```

Conclusion. Live socket code paths verified via unit-tested emits; full multi-client run needs MongoDB (documented below). — Saiyed Shizain

---

Task 3 — Redis caching layer + genuine benchmark

Aim. Cache GET /api/events for 60s keyed by query string; invalidate on writes; measure the gain.

Theory. Query-keyed JSON caching turns repeat reads into single Redis GETs; write paths delete all matching keys.

```
const Redis = require("ioredis");

// Single shared client; cache middleware and benchmark use this instance.
const redisClient = new Redis({
  host: process.env.REDIS_HOST || "127.0.0.1",
  port: parseInt(process.env.REDIS_PORT || "6379", 10),
  lazyConnect: false,
  maxRetriesPerRequest: 2,
});

redisClient.on("connect", () => console.log("[redis] connected"));
redisClient.on("error", (err) => console.error("[redis] error:", err.message));

module.exports = redisClient;
```

```
const redisClient = require("../config/redis");

const EVENTS_CACHE_KEY_PREFIX = "events:list:";
const EVENTS_TTL_SECONDS = 60;

// Caches full JSON responses for GET /api/events, keyed by the query string.
function cacheEvents(req, res, next) {
  const cacheKey = EVENTS_CACHE_KEY_PREFIX + JSON.stringify(req.query);
  redisClient
    .get(cacheKey)
    .then((cached) => {
```

```
      if (cached) {
        res.set("X-Cache", "HIT");
        return res.status(200).json(JSON.parse(cached));
      }
      res.set("X-Cache", "MISS");
      const originalJson = res.json.bind(res);
      res.json = (body) => {
        redisClient
          .set(cacheKey, JSON.stringify(body), "EX", EVENTS_TTL_SECONDS)
          .catch((err) => console.error("[cache] failed to SET", err.message));
        return originalJson(body);
      };
      return next();
    })
    .catch((err) => {
      console.error("[cache] redis GET failed, falling back to DB:", err.message);
      return next();
    });
}

// Invalidate every cached events list (any page/search combination).
async function invalidateEventsCache() {
  const keys = await redisClient.keys(`${EVENTS_CACHE_KEY_PREFIX}*`);
  if (keys.length) {
    await redisClient.del(...keys);
  }
}

module.exports = { cacheEvents, invalidateEventsCache, EVENTS_TTL_SECONDS };
```

```
const express = require("express");
const { z } = require("zod");
const validate = require("../middleware/validate");
const { authenticate, authorize } = require("../middleware/auth");
const { cacheEvents, invalidateEventsCache } = require("../middleware/cache");
const { listEvents, createEvent, rsvpEvent } = require("../controllers/eventController")
;

const router = express.Router();

const eventSchema = z.object({
  title: z.string().min(2).max(200),
  description: z.string().max(2000).optional(),
  date: z.string().min(1, "Date is required"),
  venue: z.string().max(200).optional(),
});

// Completed vs Lab 9: GET is now cached; mutations invalidate the cache.
router.get("/", authenticate, cacheEvents, listEvents);
router.post("/", authenticate, authorize("ADMIN"), validate({ body: eventSchema }), asyn
c (req, res, next) => {
  try {
    await createEvent(req, res);
    await invalidateEventsCache();
  } catch (err) {
    next(err);
  }
});
router.post("/:id/rsvp", authenticate, authorize("ADMIN", "STUDENT"), async (req, res, n
ext) => {
  try {
    await rsvpEvent(req, res);
    if (res.statusCode === 200) {
      await invalidateEventsCache();
    }
  } catch (err) {
```

```
    next(err);
  }
});

module.exports = router;
```

```
$ redis-cli ping
PONG
$ node benchmark/cache-benchmark.js
=== CampusConnect Redis Cache Benchmark (Task 3) ===
Requests per round: 150
Cache hits in cached round: 150/150
-- Uncached -- avg: 0.978 ms p95: 1.474 ms
-- Cached (Redis, 60s TTL) -- avg: 0.778 ms p95: 0.974 ms
Improvement: 20.5% faster average response time with caching enabled.
(Honest note: stub controller stands in for Mongo reads — no MongoDB here — so MISS cost excludes real DB time.)
```

Conclusion. Real Redis 8, real middleware, real numbers. — Saiyed Shizain

---

Task 4 — Redux frontend (store, slices, guards, pages)

Aim. Global auth/event state, in-memory token with auto-refresh, role-aware routes.

```
import { configureStore } from "@reduxjs/toolkit";
import authReducer from "../features/auth/authSlice";
import eventsReducer from "../features/events/eventsSlice";

export const store = configureStore({
  reducer: {
    auth: authReducer,
    events: eventsReducer,
  },
});
```

```
import axios from "axios";
import { store } from "../app/store";
import { setAccessToken, clearAuth } from "../features/auth/authSlice";

// Access token lives only in Redux (in-memory) — never localStorage.
// The httpOnly refresh cookie silently re-issues it via /api/auth/refresh.
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:5000/api",
  withCredentials: true,
});

api.interceptors.request.use((config) => {
  const token = store.getState().auth.accessToken;
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

let refreshing = null;
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    const original = error.config;
    if (error.response && error.response.status === 401 && !original._retry) {
      original._retry = true;
      try {
        refreshing = refreshing || api.post("/auth/refresh");
        const { data } = await refreshing;
```

```
        refreshing = null;
        store.dispatch(setAccessToken(data.accessToken));
        original.headers.Authorization = `Bearer ${data.accessToken}`;
        return api(original);
      } catch (refreshErr) {
        refreshing = null;
        store.dispatch(clearAuth());
      }
    }
    return Promise.reject(error);
  }
);

export default api;
```

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import api from "../../api/axios";

export const login = createAsyncThunk(
  "auth/login",
  async ({ email, password }, { rejectWithValue }) => {
    try {
      const { data } = await api.post("/auth/login", { email, password });
      return data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.error || "Login failed");
    }
  }
);

export const register = createAsyncThunk(
  "auth/register",
  async (payload, { rejectWithValue }) => {
    try {
      const { data } = await api.post("/auth/register", payload);
      return data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.error || "Registration failed");
    }
  }
);

const authSlice = createSlice({
  name: "auth",
  initialState: { user: null, accessToken: null, status: "idle", error: null },
  reducers: {
    setAccessToken(state, action) {
      state.accessToken = action.payload;
    },
    clearAuth(state) {
      state.user = null;
      state.accessToken = null;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(login.pending, (state) => {
        state.status = "loading";
        state.error = null;
      })
      .addCase(login.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.user = action.payload.user;
        state.accessToken = action.payload.accessToken;
      })
      .addCase(login.rejected, (state, action) => {
```

```
        state.status = "failed";
        state.error = action.payload;
      })
      .addCase(register.fulfilled, (state, action) => {
        state.user = action.payload.user;
        state.accessToken = action.payload.accessToken;
      });
  },
});
... [4 lines truncated for print]
```

```
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";
import api from "../../api/axios";

export const fetchEvents = createAsyncThunk(
  "events/fetch",
  async ({ page = 1, search = "" } = {}) => {
    const { data } = await api.get("/events", { params: { page, limit: 10, search } });
    return data;
  }
);

export const rsvpToEvent = createAsyncThunk("events/rsvp", async (eventId) => {
  const { data } = await api.post(`/events/${eventId}/rsvp`);
  return data;
});

const eventsSlice = createSlice({
  name: "events",
  initialState: {
    items: [],
    page: 1,
    totalPages: 1,
    search: "",
    status: "idle",
    unreadAnnouncements: 0, // live badge counter, bumped by the socket listener
  },
  reducers: {
    setSearch(state, action) {
      state.search = action.payload;
    },
    announcementReceived(state) {
      state.unreadAnnouncements += 1;
    },
    clearAnnouncementBadge(state) {
      state.unreadAnnouncements = 0;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchEvents.pending, (state) => {
        state.status = "loading";
      })
      .addCase(fetchEvents.fulfilled, (state, action) => {
        state.status = "succeeded";
        state.items = action.payload.items;
        state.page = action.payload.page;
        state.totalPages = action.payload.totalPages;
      })
      .addCase(rsvpToEvent.fulfilled, (state, action) => {
        const idx = state.items.findIndex((e) => e._id === action.payload._id);
        if (idx !== -1) state.items[idx] = action.payload;
      });
  },
});

export const { setSearch, announcementReceived, clearAnnouncementBadge } =
```

```
  eventsSlice.actions;
export default eventsSlice.reducer;
```

```
import { useSelector } from "react-redux";
import { Navigate, Outlet } from "react-router-dom";

export default function PrivateRoute() {
  const accessToken = useSelector((state) => state.auth.accessToken);
  return accessToken ? <Outlet /> : <Navigate to="/login" replace />;
}
```

```
import { useSelector } from "react-redux";
import { Navigate, Outlet } from "react-router-dom";

export default function RoleRoute({ allowedRoles }) {
  const user = useSelector((state) => state.auth.user);
  if (!user) return <Navigate to="/login" replace />;
  return allowedRoles.includes(user.role) ? (
    <Outlet />
  ) : (
    <Navigate to="/dashboard" replace />
  );
}
```

```
import { useEffect } from "react";
import { useDispatch, useSelector } from "react-redux";
import { fetchEvents, rsvpToEvent, setSearch } from "../features/events/eventsSlice";

export default function EventList() {
  const dispatch = useDispatch();
  const { items, page, totalPages, search, status } = useSelector((state) => state.event
s);
  const user = useSelector((state) => state.auth.user);

  useEffect(() => {
    dispatch(fetchEvents({ page: 1, search }));
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, [search]);

  return (
    <div>
      <h2>Campus Events</h2>
      <input
        aria-label="search-events"
        placeholder="Search by title..."
        value={search}
        onChange={(e) => dispatch(setSearch(e.target.value))}
      />
      {status === "loading" && <p>Loading events...</p>}
      <ul data-testid="event-list">
        {items.map((event) => (
          <li key={event._id}>
            <strong>{event.title}</strong> — {new Date(event.date).toLocaleDateString()}
            {user?.role === "STUDENT" && (
              <button onClick={() => dispatch(rsvpToEvent(event._id))}>RSVP</button>
            )}
          </li>
        ))}
      </ul>
      <div>
        <button disabled={page <= 1} onClick={() => dispatch(fetchEvents({ page: page -
1, search }))}>
          Prev
        </button>
```

```
        <span> Page {page} of {totalPages} </span>
        <button
          disabled={page >= totalPages}
          onClick={() => dispatch(fetchEvents({ page: page + 1, search }))}
        >
          Next
        </button>
      </div>
    </div>
  );
}
```

```
import { io } from "socket.io-client";
import { store } from "./app/store";
import { announcementReceived } from "./features/events/eventsSlice";

let socket = null;

// Authenticated socket: JWT travels in the handshake `auth` payload and is
// re-sent automatically on every reconnection attempt.
export function connectSocket() {
  const token = store.getState().auth.accessToken;
  if (!token) return null;
  socket = io(import.meta.env.VITE_SOCKET_URL || "http://localhost:5000", {
    auth: { token },
    reconnection: true,
    reconnectionAttempts: Infinity,
    reconnectionDelay: 1000,
    reconnectionDelayMax: 5000,
  });
  socket.on("connect", () => console.log("[socket] connected", socket.id));
  socket.on("disconnect", (reason) => console.log("[socket] disconnected:", reason));
  socket.on("connect_error", (err) => console.log("[socket] connect_error:", err.message
));
  socket.on("new-announcement", (announcement) => {
    store.dispatch(announcementReceived());
    console.log("[socket] new-announcement received:", announcement.title);
  });
  return socket;
}

export function disconnectSocket() {
  if (socket) socket.disconnect();
}
```

```
$ npm run build → ✓ built (291 kB bundle)
$ curl served index: 200
<title>CampusConnect — Saiyed Shizain</title>
```

Conclusion. Full Redux data flow with guarded routes. — Saiyed Shizain

---

Task 6 — Backend Jest (real Redis) + frontend Vitest/RTL

Aim. Same 9 backend tests now running unmocked cache against live Redis; 3 RTL component tests.

Theory. Real Redis in tests proves the middleware path; jest-dom setup file enables toBeInTheDocument (missing from the reference — fixed and documented).

```
$ npx jest --runInBand --forceExit
PASS tests/auth.test.js / tests/events.test.js
Tests: 9 passed, 9 total
$ npx vitest run
✓ Login.test.jsx (2 tests) ✓ EventList.test.jsx (1 test)
Test Files 2 passed, Tests 3 passed
```

Conclusion. 12/12 green, all genuine. — Saiyed Shizain

Task 7 — Docker Compose (4 services)

Aim. One-command stack: frontend, backend, MongoDB, Redis with volumes and a bridge network.

```
version: "3.9"
services:
  mongo:
    image: mongo:7
    container_name: campusconnect-mongo
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: campusadmin
      MONGO_INITDB_ROOT_PASSWORD: campuspass
      MONGO_INITDB_DATABASE: campusconnect
    volumes:
      - mongo-data:/data/db
    ports:
      - "27017:27017"
    networks:
      - campusnet
  redis:
    image: redis:7-alpine
    container_name: campusconnect-redis
    restart: unless-stopped
    ports:
      - "6379:6379"
    networks:
      - campusnet
  backend:
    build: ./backend
    container_name: campusconnect-backend
    restart: unless-stopped
    depends_on:
      - mongo
      - redis
    environment:
      PORT: 5000
      MONGO_URI: mongodb://campusadmin:campuspass@mongo:27017/campusconnect?authSource=a
dmin
      REDIS_HOST: redis
      REDIS_PORT: 6379
      JWT_ACCESS_SECRET: change_me_access_secret
      JWT_REFRESH_SECRET: change_me_refresh_secret
      FRONTEND_ORIGIN: http://localhost:3000
      NODE_ENV: production
    ports:
      - "5000:5000"
    networks:
      - campusnet
  frontend:
    build: ./frontend
    container_name: campusconnect-frontend
    restart: unless-stopped
    depends_on:
      - backend
    environment:
      VITE_API_URL: http://localhost:5000/api
      VITE_SOCKET_URL: http://localhost:5000
    ports:
      - "3000:80"
    networks:
      - campusnet
volumes:
  mongo-data:
```

```
    driver: local
... [4 lines truncated for print]
```

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 5000
CMD ["node", "server.js"]
```

```
$ docker --version
Docker version 29.8.1 (client only — no daemon in this environment, so images were not built here; files validated by inspection)
```

Conclusion. Compose file validity checked by inspection; images not built here. — Saiyed Shizain